

VELORA — Module POS (Part 3)

Services · Controllers · Routes

10. Services

app/Modules/POS/Services/ProductService.php

```
<?php
```

```
namespace App\Modules\POS\Services;
```

```
use App\Modules\POS\DTOs\CreateProductDTO;
```

```
use App\Modules\POS\Models\Product;
```

```
use App\Modules\POS\Repositories\Contracts\ProductRepositoryInterface;
```

```
use App\Modules\Subscription\Services\SubscriptionService;
```

```
use App\Shared\Exceptions\VeloraException;
```

```
use App\Shared\Services\BaseService;
```

```
use Illuminate\Pagination\LengthAwarePaginator;
```

```
use Illuminate\Support\Facades\DB;
```

```
use Illuminate\Support\Facades\Storage;
```

```
class ProductService extends BaseService
```

```
{
    public function __construct(
        private readonly ProductRepositoryInterface $productRepo,
        private readonly SubscriptionService $subscriptionService
    ) {}

    public function getAll(string $businessId, array $filters = []): LengthAwarePaginator
    {
        return $this->productRepo->findByBusiness($businessId, $filters);
    }

    public function search(string $businessId, string $query): LengthAwarePaginator
    {
        return $this->productRepo->search($businessId, $query);
    }

    public function findOrFail(string $id): Product
    {
        $product = $this->productRepo->findById($id);

        if (!$product || $product->business_id !== app('current.business_id')) {
            throw VeloraException::notFound('Product');
        }
    }
}
```

```

        return $product;
    }

    public function create(CreateProductDTO $dto): Product
    {
        // 1. Check feature limit (max_products)
        $currentCount = $this->productRepo->countByBusiness($dto->businessId);

        if (!$this->subscriptionService->checkLimit($dto->businessId, 'max_products', $currentCount)) {
            throw new VeloraException(
                'Batas maksimal produk telah tercapai. Upgrade paket untuk menambah produk.',
                403,
                'SUBSCRIPTION_LIMIT_REACHED'
            );
        }

        // 2. Wrap in transaction
        return DB::transaction(function () use ($dto) {
            $productData = $dto->toArray();
            unset($productData['variants']);

            $product = $this->productRepo->create($productData);

            // 3. Create variants if needed
            if ($dto->hasVariants && !empty($dto->variants)) {
                foreach ($dto->variants as $index => $variantData) {
                    $this->productRepo->createVariant([
                        'product_id'      => $product->id,
                        'business_id'     => $product->business_id,
                        'name'             => $variantData['name'],
                        'sku'              => $variantData['sku'] ?? ($product->sku . '-' . ($index + 1)),
                        'barcode'         => $variantData['barcode'] ?? null,
                        'cost_price'      => $variantData['cost_price'] ?? $product->cost_price,
                        'selling_price'   => $variantData['selling_price'] ?? $product->selling_price,
                        'stock_quantity' => $variantData['stock_quantity'] ?? 0,
                        'attributes'      => $variantData['attributes'] ?? null,
                    ]);
                }
            }

            return $product->fresh(['category', 'unit', 'variants']);
        });
    }

```

```
}

public function uploadImage(string $id, \Illuminate\Http\UploadedFile $file): Product
{
    $product = $this->findOrFail($id);

    if ($product->image) {
        Storage::disk('public')->delete($product->image);
    }

    $path = $file->store("businesses/{$product->business_id}/products/{$id}", 'public');

    return $this->productRepo->update($id, ['image' => $path]);
}

public function delete(string $id): void
{
    $product = $this->findOrFail($id);
    $this->productRepo->delete($product->id);
}
}
```

11. Controllers

app/Modules/POS/Controllers/ProductController.php

```
<?php
```

```
namespace App\Modules\POS\Controllers;
```

```
use App\Modules\POS\DTOs\CreateProductDTO;
```

```
use App\Modules\POS\Requests\StoreProductRequest;
```

```
use App\Modules\POS\Resources\ProductResource;
```

```
use App\Modules\POS\Services\ProductService;
```

```
use App\Shared\Controllers\BaseController;
```

```
use Illuminate\Http\JsonResponse;
```

```
use Illuminate\Http\Request;
```

```
class ProductController extends BaseController
```

```
{
```

```
    public function __construct(private readonly ProductService $service) {}
```

```
    public function index(Request $request): JsonResponse
```

```
    {
```

```
        $businessId = app('current.business_id');
```

```
        $paginator = $this->service->getAll($businessId, $request->all());
```

```
        return $this->paginated($paginator, ProductResource::collection($paginator));
```

```
    }
```

```
    public function store(StoreProductRequest $request): JsonResponse
```

```
    {
```

```
        $businessId = app('current.business_id');
```

```
        $product = $this->service->create(
```

```
            CreateProductDTO::fromRequest($request->validated(), $businessId)
```

```
        );
```

```
        return $this->created(new ProductResource($product), 'Produk berhasil ditambahkan.');
```

```
    }
```

```
    public function show(string $id): JsonResponse
```

```
    {
```

```
        $product = $this->service->findOrFail($id);
```

```
        return $this->success(new ProductResource($product));
```

```

}

public function search(Request $request): JsonResponse
{
    $businessId = app('current.business_id');
    $query = $request->query('q', '');

    if (empty($query)) {
        return $this->success([]);
    }

    $results = $this->service->search($businessId, $query);
    return $this->paginated($results, ProductResource::collection($results));
}

public function uploadImage(Request $request, string $id): JsonResponse
{
    $request->validate([
        'image' => ['required', 'image', 'mimes:jpg,jpeg,png,webp', 'max:2048'],
    ]);

    $product = $this->service->uploadImage($id, $request->file('image'));
    return $this->success(new ProductResource($product), 'Foto produk berhasil diupload.');
```

```

}

public function destroy(string $id): JsonResponse
{
    $this->service->delete($id);
    return $this->noContent();
}
}

```

12. Routes

app/Modules/POS/Routes/api.php

```
<?php
```

```
use App\Modules\POS\Controllers\CategoryController;
use App\Modules\POS\Controllers\ProductController;
use App\Modules\POS\Controllers\UnitController;
use Illuminate\Support\Facades\Route;

Route::middleware(['auth:sanctum', 'tenant.scope', 'subscription.active'])->group(function () {

    // Categories
    Route::apiResource('categories', CategoryController::class);
    Route::get('categories-tree', [CategoryController::class, 'tree'])->name('categories.tree');

    // Units
    Route::apiResource('units', UnitController::class);

    // Products
    Route::get('products/search', [ProductController::class, 'search'])->name('products.search');
    Route::post('products/{product}/image', [ProductController::class, 'uploadImage'])->name('products.uploadImage');
    Route::apiResource('products', ProductController::class);

});
```

Checklist Module POS

- ✓ Migrations: categories , units , products , product_variants , product_images
- ✓ Enums: ProductStatus , ProductType
- ✓ Models: Category , Unit , Product , ProductVariant , ProductImage
- ✓ Repository Interface + Implementation (Category, Unit, Product)
- ✓ DTOs: CreateProductDTO
- ✓ Form Requests: StoreProductRequest dengan validasi variant jika has_variants=true
- ✓ Resources: ProductResource , ProductVariantResource

- ✓ Services: ProductService (dengan pengecekan `max_products` subscription limit dan DB transaction untuk variants)
- ✓ Controllers: ProductController
- ✓ Routes: CRUD products, search, upload-image

Semua fondasi utama untuk POS (Products) telah disiapkan. Module lain seperti Inventory dan Sales akan menggunakan Model `Product` dan `ProductVariant` ini.